

YuloStores — Cross-Portal Technical Audit

Prepared 2026-09-08 • Scope: customer app, backend API, restaurant portal, super-admin portal

Repositories reviewed: yulostores_customer_app , yulo_backend , yulo_restaurant , yulo_super_admin

Branch: main (customer app had uncommitted working-tree changes at review time)

5
Critical

5
High

8
Medium

4
Low / cleanup

Do not ship to production until these are resolved: C1, C2, C3, C4, C5, H1, H2, H5. Every one of them is a config or wiring issue — none require large code changes. Several are deliberately-temporary states the team already flagged in code comments; this report collects them in one place so nothing is forgotten at cut-over.

How to read this

Severity	Meaning
CRITICAL	Causes data loss, a security hole, money handled incorrectly, or a totally non-functional build. Fix before any production/beta release.
HIGH	A core user-facing feature is broken or a whole portal fails in a realistic deployment. Fix before release.
MEDIUM	Real defect or risk, but with a workaround or limited blast radius. Schedule soon.
LOW	Cleanup, stale comments, hygiene. No user impact.

Locations are given as repo / path:line . Line numbers are from the review-time checkout and may drift by a few lines.

1 Critical

C1 **CRITICAL** Placeholder JWT secrets and plaintext third-party credentials in the backend .env

Where `yulo_backend / server/.env:4-6, 10-17, 20`

What

- `JWT_ACCESS_SECRET`, `JWT_REFRESH_SECRET`, `JWT_STAFF_SECRET` are literally `dev_access_secret_replace_in_production_32c` etc. They satisfy the `min(32)` check in `config/env.js` so nothing complains, but anyone who has seen this repo can mint a valid access / staff token for any account (including `admin` and `restaurant_owner`).
- Live-looking service credentials sit in cleartext in the working tree: MongoDB Atlas (`admin_db_user:rgVqfZZZ...`), Redis (`default:fp8U40aH...`), Cloudinary API secret, and the MessageCentral key (`eXVsb3N0b3J1c1Byb2RAMTizNA==` decodes to `yulostoresProd@1234`).

Impact Full authentication bypass and account takeover if these secrets reach a live environment; credential exposure for four external services; a weak, guessable SMS-provider password.

Fix

1. Generate real 48-byte secrets (`node -e "console.log(require('crypto').randomBytes(48).toString('base64url'))"`) and set all four `JWT_*_SECRET` values only in the DigitalOcean App Platform dashboard, never in a file.
2. Rotate the MongoDB, Redis, Cloudinary and MessageCentral credentials that have been in this file, and give MessageCentral a strong password.
3. Add a startup guard in `config/env.js`: if `NODE_ENV === 'production'` and any secret matches `/replace_in_production|dev.*_secret/`, `process.exit(1)`.
4. Confirm this `.env` is genuinely local-only — it is correctly gitignored (verified), but treat every value in it as compromised.

C2 **CRITICAL** OTP login is fully bypassed — any 6-digit code signs in as any phone number

Where `yulo_backend / server/.env:20 ( SMS_PROVIDER="bypass" )` • logic in `server/services/otp.service.js:59-66, 146-150`

What With `SMS_PROVIDER=bypass`, `verifyOtp()` accepts `/^\d{6}$/` for any number that has requested a code — no SMS, no stored hash check. The customer app mirrors this expectation (`yulostores_customer_app / src/services/auth.ts:8-12`). The code comments say this is temporary because the MessageCentral balance is exhausted.

Impact Anyone can sign in as any customer by entering their phone number and any 6 digits. Full account access (addresses, order history, saved payment labels, support tickets).

Fix Top up MessageCentral and set `SMS_PROVIDER=messagecentral` (env-only, no code change). Until then, treat every customer session as unauthenticated. Consider a short-lived allow-list of test numbers instead of a global bypass. Add the same production guard as C1 so `bypass` can never be the value when `NODE_ENV=production`.

C3 **CRITICAL** Online payments are simulated — orders are marked `paid` with no money collected

Where `yulo_backend / server/.env` (no `RAZORPAY_*` keys present) •
`server/controllers/order.controller.js:234-254` (`simulatePayment`) • `yulostores_customer_app / src/services/payments.ts:405-466` (`razorpayGateway` is a stub that throws)

What No Razorpay credentials are configured, so `createRazorpayOrder()` returns `null`, the app falls back to `POST /api/orders/:id/payment/simulate`, and because `env.RAZORPAY_KEY_ID` is unset that endpoint does *not* 400 — it sets `paymentStatus = 'paid'` with a fake `sim_...` id. Separately, the real client gateway is not implemented: `razorpayGateway.pay()` immediately throws “Online payments aren’t enabled in this build yet.”

Impact If the customer app is used for real: every “online” order is fulfilled without payment. If a Razorpay key is added without also finishing the client integration: every online order fails at the pay step. The backend only logs a warning in this state (`server/index.js:30-40`).

Fix Decide the launch payment posture:

- **COD-only launch:** hide/disable the online methods in `src/services/payments.ts` so nothing routes to the simulated gateway in a shipped build.
- **Online launch:** set `RAZORPAY_KEY_ID/SECRET/WEBHOOK_SECRET`, then complete step 3 in the `payments.ts` header (`npm install react-native-razorpay`, remove the `PaymentError` throw, uncomment the block), and test the `/payment/verify` + `/api/webhooks/razorpay` paths end to end.

The backend `verify/webhook` code (`server/services/payment.service.js`) is already implemented and looks correct — only keys + the client SDK are missing.

C4 **CRITICAL** Customer app has no production API URL — a release build cannot reach the backend

Where `yulostores_customer_app / app.json:84` (`"apiUrl": null`) • resolution in `src/constants/Api.ts:71-92`

What `resolveBaseUrl()` priority is: `extra.apiUrl` → Expo dev-server LAN IP → `http://10.0.2.2:3000` → `http://localhost:3000`. In a standalone (EAS) build there is no dev-server host, so it resolves to `http://10.0.2.2:3000 / localhost` — unreachable from a real device. It is also cleartext `http://`, which Android blocks by default.

Impact Every screen in a shipped APK/IPA fails to load. The app only works today because it is run through Expo dev tooling.

Fix Set `expo.extra.apiUrl` to the HTTPS API origin (e.g. `https://api.yulostores.in`) before `eas build`, or wire it from an EAS build profile / env var. Also add the Android `usesCleartextTraffic` exception only if a non-HTTPS staging URL is ever needed. Do the same review for `expo.extra` on every build profile.

C5 **CRITICAL** Order rating always fails with 404 — endpoint path mismatch

Where caller `yulostores_customer_app / src/services/orders.ts:273` → `POST /api/orders/${orderId}/review`
used by `yulostores_customer_app / app/order/[id]/track.tsx:289` (`submitReview`)
backend route is `yulo_backend / server/routes/review.routes.js:8` mounted at `/api/reviews` (`server/app.js`) → real path is `POST /api/reviews/:orderId/review`

What The customer app posts to `/api/orders/:id/review`, which does not exist (`server/routes/order.routes.js` has no `/review`). The restaurant portal calls the correct path (`yulo_restaurant / src/api/customer.api.js` → `/reviews/${orderId}/review`), confirming the backend contract.

Impact Customers can never rate a delivered order from the app; the rating sheet on the tracking screen always errors.

Fix One line in `src/services/orders.ts:273`: change the path to `/api/reviews/${orderId}/review`. Also align the doc comment on `submitReview` — the backend's not-delivered code is `ORDER_NOT_DELIVERED`, not the phrasing the comment implies.

2 High

H1 **HIGH** Android Google Maps API key is a placeholder

Where `yulostores_customer_app / app.json:37` → `"apiKey": "YOUR_ANDROID_GOOGLE_MAPS_API_KEY"`

What `expo-maps` is used in the location flow (`src/components/location/MapCanvas.tsx`, `app/location/map.tsx`). The Google provider on Android needs a valid Maps SDK key; with the placeholder the map renders blank / grey tiles on a device build.

Impact “Set location on map” and delivery-pin confirmation are unusable on Android release builds. iOS uses Apple Maps and is unaffected.

Fix Create an Android Maps SDK key in Google Cloud (restrict by SHA-1 + package `com.yulostores.customerapp`), put it in `app.json` or an EAS secret, and rebuild. Verify billing is enabled on the GCP project.

H2 **HIGH** `ALLOWED_ORIGINS` is localhost-only — a real browser deploy is CORS-blocked

Where `yu1o_backend / server/.env:13` • usage `server/app.js:52-56`

What Current value is

`http://localhost:5174,http://localhost:8081,http://localhost:8082,http://localhost:5173` . `cors()` exact-matches the request `Origin` against this list (with `credentials:true`), so any deployed portal on a Vercel/custom domain is rejected.

Impact The restaurant portal and super-admin portal cannot talk to the API once deployed — every request fails preflight, including auth. (Does not affect the mobile app or `curl`, which is why it can pass casual testing.)

Fix In the production backend env, set `ALLOWED_ORIGINS` to the exact list of deployed portal origins (restaurant, super-admin, and the guest menu app if browser-based). Keep it out of the committed file. Confirm Socket.IO CORS uses the same list (`server/socket.js`).

H3 **HIGH** Push notifications are not implemented anywhere

Where `yu1o_backend / server/services/notify.service.js` (Socket.IO room emits only) • dead endpoint `POST/DELETE /api/users/me/devices` (`server/routes/user.routes.js:131-134`) + `server/models/UserDevice.js` • `yu1ostores_customer_app / src/services/notifications.ts` is effectively empty; `expo-notifications` is in `package.json` + configured in `app.json` plugins but never imported

What There is no FCM / APNs / Expo Push integration (no `expo-server-sdk` or `firebase-admin` in the backend deps). Order-status updates reach the customer only while a screen holds an open socket (e.g. the live tracking screen). The device-token registration route and model exist but nothing writes to them and nothing reads them to send a push.

Impact Customers get no notification when the app is backgrounded or closed — no “order confirmed”, “out for delivery”, “delivered”. For a delivery product this is a major expectation gap. The `expo-notifications` dependency and iOS/Android permission prompts ship for no functional benefit.

Fix Scope a real push implementation: register the Expo push token from the app on sign-in via the existing `/api/users/me/devices` route, and have `notify.service.js` also fan out to `expo-server-sdk` for the customer-facing events. If push is explicitly out of scope for launch, remove `expo-notifications`, the plugin config, and the permission strings so the app stops asking.

H4 **HIGH** Two separate Admin portals against the same `/api/admin/*` API

Where `yulo_super_admin` (standalone Vite app) • `yulo_restaurant / src/screens/admin/* + src/api/admin.api.js + src/context/AdminAuthContext.jsx`, mounted at `/admin/*` in `yulo_restaurant / src/App.jsx:79`

What The restaurant portal contains a full second admin UI (login, stores, customers, tickets, dashboard, reports) that overlaps the dedicated `yulo_super_admin` project. `yulo_super_admin` is the superset — it also has Finance, Bills, and Delivery-Partner management the restaurant copy lacks.

Impact Divergence risk: two auth contexts, two client layers, two sets of screens to keep in step with backend changes. Whoever fixes an admin bug may fix it in the wrong copy. Also a larger restaurant-portal bundle and attack surface.

Fix Confirm `yulo_super_admin` is canonical, then delete `src/screens/admin/`, `src/api/admin.api.js`, `src/context/AdminAuthContext.jsx`, `src/components/AdminRoute.jsx` and the `/admin/*` routes from the restaurant portal. If the embedded one is intentionally kept for a subset of admins, document why and freeze feature work on it.

H5 **HIGH** `REFRESH_COOKIE_SAMESITE = none` with `NODE_ENV=development` silently logs portal users out

Where `yulo_backend / server/.env:1, 22` • logic `server/utils/refreshCookie.js:44-53`

What `sameSite:'none'` forces `secure:true`. On plain-`http://localhost` the browser refuses to store the refresh cookie. Sign-in appears to work, then when the 15-minute access token expires every screen 401s and the next reload ends the session — exactly the footgun the `.env.example` comment warns about.

Impact Broken local dev for the owner / staff / super-admin portals (mobile app is unaffected — it stores its refresh token in SecureStore, not a cookie). Wastes developer time chasing “random” logouts.

Fix Remove line 22 from the local `.env` (let `config/env.js` resolve it — `1ax` in dev, `none` in prod), or set it to `1ax` for local work. In production it must be `none` because the portals and API are cross-site. Also note `OTP_MAX_REQUESTS_PER_WINDOW=10` on line 21 is loosened from the prod-safe default of 3 — reset before launch.

3 Medium

M1 **MEDIUM** `@sentry/node` is a dependency but is never initialized

Where `yulo_backend / server/package.json ( "@sentry/node": "^10.58.0" );` no `Sentry.init` / import anywhere in `server/`.

Impact No server-side error monitoring despite the dependency being installed — a false sense of coverage. Unused package in the deploy.

Fix Either initialize Sentry in `server/index.js` before route registration (DSN from env) and add its Express error handler, or drop the dependency.

M2 MEDIUM Customer app has no crash / error reporting

Where `yulostores_customer_app / src/lib/logger.ts:90-96, 951-957` — `reportError()` only calls `console.error`; the crash-reporter hook is a `TODO(observability)`.

Impact Production crashes and handled errors are invisible to the team. The infrastructure is in place (single `reportError` seam, global handler installer) — only the SDK call is missing.

Fix Add Sentry React Native (or Crashlytics) and wire it into `reportError` and `installGlobalErrorLogging`.
Low effort given the existing structure.

M3 MEDIUM The “Scan” tab is a dead end

Where `yulostores_customer_app / app/(tabs)/scan.tsx:42-45`

What `handleBarcodeScanned` shows `Alert.alert('QR Code Scanned!', 'Data: ' + data)` and stops. The QR payload is never parsed and nothing navigates. The tab subtitle promises “Scan a table QR to order directly”.

Impact A primary bottom-tab feature does nothing. Combined with the fact that customer dine-in ordering has no backend support (see Known Gaps §5, item 1.4), the whole QR path is unbuilt.

Fix Decide whether dine-in QR ordering is in scope. If not, remove or hide the Scan tab. If yes, this needs backend work first (accept `type: "dine_in" + tableId` on `POST /api/orders`), then parse the QR deep link and route into the menu.

M4 MEDIUM Backend does not pin a Node.js version

Where `yulo_backend / server/package.json` — no `engines` field. `server/index.js:12-20` explicitly depends on Node 20+ behaviour (guards `net.setDefaultAutoSelectFamily`).

Impact The deploy platform can silently pick a different Node major across builds. The IPv6/Happy-Eyeballs workaround in `index.js` exists precisely because a version difference caused Cloudinary reads to stall for 60s; an older runtime would reintroduce that class of bug.

Fix Add `"engines": { "node": ">=20 <23" }` (or the exact version DigitalOcean runs) and an `.nvmrc`. Pin it in the App Platform settings too.

M5

MEDIUM

Payment-method and store-settings catalogues are hand-copied between backend and frontend

Where `yulostores_customer_app / src/services/payments.ts:312-327` is a manual twin of `yulo_backend / server/config/appConfig.config.js`

`yulo_restaurant / src/lib/fieldRules.js` is a manual twin of `yulo_backend / server/config/storeSettings.config.js`

What Both are documented as “synchronous twins, keep in step by hand.” The customer app can’t block checkout on a config fetch, and the portal renders the settings form from local rules.

Impact Silent drift: add a payment method or change a validation rule on the backend and the client keeps rendering the old set until someone remembers to edit the twin. No test catches it.

Fix Add a lightweight contract test (or a CI script) that fetches `GET /api/app/config` and `GET /api/owner/settings-requirements` and diffs the shapes/keys against the committed twins, failing the build on divergence. Or generate the twins from a shared JSON at build time.

M6

MEDIUM

Customer app `.env` is git-tracked

Where `yulostores_customer_app / .env` (tracked, currently empty) • `.gitignore` only excludes `.env*.local`

Impact Any value a developer adds to `.env` for local work gets committed by default. Today it is empty so there is no leak, but the setup invites one.

Fix `git rm --cached .env`, add `.env` to `.gitignore`, and commit a `.env.example` instead. (Runtime config for this app actually lives in `app.json` → `extra`, so `.env` may not be needed at all — confirm and delete if so.)

M7

MEDIUM

Offline “local session” auth fallback in the customer app

Where `yulostores_customer_app / src/constants/config.ts:135` (`AUTH_OTP_BYPASS`, currently `false`) • `src/services/auth.ts:110-170`

What When `AUTH_OTP_BYPASS` is `true`, a failed backend verify mints a fake `{ _id: 'local:<phone>', role: 'customer' }` session client-side. It is off now, but the path exists and is easy to flip on for a demo and forget.

Impact If enabled in a shipped build, the app shows a “signed in” state with no server session; every authenticated call 401s and screens show empty/needs-sign-in states inconsistently.

Fix Keep it `false` for every real build. Add a build-time assertion (or a lint rule) that fails a production build if `AUTH_OTP_BYPASS !== false`. Remove once C2 is resolved.

M8 **MEDIUM** No automated tests in any of the four repos

Where `yulo_backend / server/package.json` → `"test": "echo \"Error: no test specified\" && exit 1";` no test files or runners in the customer app or either portal.

Impact Regressions like C5 (a one-word path typo that breaks a feature) ship undetected. Refactors are high-risk. The API contract between four codebases is only enforced by hand and by the prose in `API.md` / `API-GAPS.md`.

Fix Start with the highest-leverage layer: a handful of supertest integration tests over the backend auth + order + cart + checkout routes, and a contract test (M5) for the shared config. Add a CI workflow that runs them on PRs.

4 Low / cleanup

L1 **LOW** Stale doc comments

Where `yulostores_customer_app / src/components/CartBar.tsx:22-24` — says “neither the cart screen nor a restaurant menu route exists yet”; both exist and the buttons navigate (`router.push('/cart')` , `router.push('/restaurant/...')`).

`src/services/checkout.ts` header and `src/services/orders.ts:262-270` describe behaviour that has since changed.

Fix Trim the comments to match current behaviour during the next touch of these files. No functional impact.

L2 **LOW** `console.*` used directly in backend bootstrap

Where `yulo_backend / server/config/db.js, config/redis.js, config/env.js` use `console.log/warn/error` instead of the pino `logger`.

Impact These lines run before/around logger setup so it is defensible, but the output skips pino’s JSON formatting and log levels — slightly harder to parse in production log aggregation.

Fix Move to `logger` where it is available; leave the pre-logger `env.js` failure path as-is.

L3 **LOW** Ad-hoc migration scripts with no runner or tracking

Where `yulo_backend / server/scripts/` — 9 one-off `backfill*/fix*` scripts.

Impact No record of which have been run against which environment; risk of double-run or missed-run during the next deploy or a fresh environment bring-up.

Fix Adopt a minimal migration tool (e.g. `migrate-mongo`) or at least a `migrations_applied` collection and a README listing run order and idempotency of each script.

L4 LOW Restaurant portal local .env bypasses the documented dev proxy

Where `yu_lo_restaurant / .env` → `VITE_API_BASE = http://localhost:3000` (the `.env.example` and `vite.config.js` comments say leave it blank to use the Vite proxy).

Impact Local requests go cross-origin to `:3000` instead of same-origin through the proxy. Works today (localhost is same-site, and `:3000` is in `ALLOWED_ORIGINS`), but it is a deviation from the intended setup and interacts badly with H5.

Fix Blank the value locally, or document why the direct connection is wanted. `.env` is gitignored here so this is per-developer.

5 Pre-existing known gaps (already catalogued by the team)

`yu_lo_restaurant / API-GAPS.md` is a thorough, current list of features that are designed but not backed by an API, or where the frontend works around an ambiguous contract. It is worth reading in full. The load-bearing items:

#	Gap	State
1.1	Customer assistance requests (call waiter / water / bill / cleaning) — <i>partially closed</i> : guest request routes now exist (<code>/api/restaurants/:id/requests</code>), but staff-queue endpoints are still thin.	Mostly resolved
1.2	Customer-facing offers list — no public “active discounts” endpoint. <i>Note</i> : the customer app <i>does</i> call <code>POST /api/cart/apply-promo</code> , which exists, so coupon entry works even though browsing offers does not.	Open (browse), Works (redeem)
1.3	Password change — no route; <code>PATCH /api/users/me</code> covers name/phone/avatar only.	Open
1.4	Customer dine-in ordering — <code>POST /api/orders</code> requires <code>type:"delivery"</code> . QR self-ordering from the customer app is not possible; a waiter must place it. Blocks M3.	Open (biggest)
1.7–1.9	Admin: platform activity log, cross-restaurant order/offer monitoring, QR oversight, system settings, staff & roles listing — screens removed because nothing backs them.	Open
1.10–1.12	Menu add-ons/modifiers, ingredient cost/inventory, cancellation reasons & refunds — data models don’t support the designed UI.	Open
2	No <code>manager</code> role — the manager portal at <code>/manager/*</code> runs on the owner session against owner-scoped endpoints.	Open (by design for now)
4.11	Socket.IO is largely unconsumed by the portals — most live data is polled. Wiring the existing <code>new_order</code> / <code>order_status_updated</code> / <code>live_stats</code> events would cut polling load.	Open
4.13	Compliance fields (<code>panNumber</code> , <code>gstNumber</code> , ...) are locked in the portal UI after first save but the server still accepts changes — the rule lives only in the client.	Open (server-side enforcement)

6 Quick-fix checklist

ID	One-line action	Owner	Effort
C1	Rotate all leaked service creds; set strong <code>JWT_*_SECRET</code> in the platform dashboard; add prod guard in <code>env.js</code>	Backend / DevOps	M
C2	Top up MessageCentral, set <code>SMS_PROVIDER=messagecentral</code> ; guard against <code>bypass</code> in prod	Backend / Ops	S
C3	Pick COD-only vs. real Razorpay; either disable online methods in <code>payments.ts</code> or finish the SDK integration + set keys	Mobile + Backend	M-L
C4	Set <code>expo.extra.apiUrl</code> to the HTTPS API origin for every build profile	Mobile	S
C5	Change one path string in <code>src/services/orders.ts:273</code> to <code>/api/reviews/\${orderId}/review</code>	Mobile	XS
H1	Add a real restricted Android Google Maps key	Mobile / DevOps	S
H2	Set production <code>ALLOWED_ORIGINS</code> to the deployed portal domains	DevOps	XS
H3	Implement Expo push (register token + fan out from <code>notify.service.js</code>), or remove <code>expo-notifications</code>	Mobile + Backend	M-L
H4	Delete the embedded admin portal from <code>yulo_restaurant</code> (confirm <code>yulo_super_admin</code> is canonical first)	Frontend	S
H5	Remove <code>REFRESH_COOKIE_SAMESITE</code> + reset <code>OTP_MAX_REQUESTS_PER_WINDOW</code> in local/prod env as appropriate	Backend / Ops	XS
M1–M2	Initialize Sentry on backend and in the customer app (or drop the unused dep)	All	S
M3	Hide the Scan tab, or scope dine-in ordering properly (needs 1.4 first)	Mobile + Backend	S / L
M4	Add <code>engines</code> + <code>.nvmrc</code> to the backend and pin the platform runtime	DevOps	XS
M5 / M8	Add a config-contract test and a small backend integration suite in CI	All	M
M6	<code>git rm --cached</code> the customer app <code>.env</code> ; ignore it	Mobile	XS
M7	Assert <code>AUTH_OTP_BYPASS === false</code> at build time	Mobile	XS

7 Method & coverage

This pass was a static read of all four repositories: configuration and env files, the backend route/controller/service tree, the customer app service + hook layer, and the two portals' API clients and routing. Cross-portal API contracts were checked by extracting every endpoint string each frontend calls and matching it against the backend's mounted routes (that is how C5 surfaced). It did **not** include: running the apps, load/perf testing, a dependency CVE scan, a full read of every screen

component, or a deep review of the delivery-partner backend surface (/api/partner/* exists and is substantial but no partner app was in scope). A follow-up focused on the partner flow and on database index / query-cost review would be worthwhile.

End of report — 22 findings (5 critical, 5 high, 8 medium, 4 low). Generated from a working-tree review on 2026-09-08.